# ConveyorVision — Bản điều chỉnh đề xuất: dữ liệu, chia tập và kế hoạch thực thi

Phiên bản 1.1 | 15/10/2026 | Điều chỉnh từ phiên bản 1.0 (11/10/2026)

## 1. Tóm tắt thay đổi so với v1.0

Bản v1.0 giả định tự thu thập và tự gán nhãn dữ liệu. Bản v1.1 chuyển sang dùng hai bộ dữ liệu công khai đã gán nhãn sẵn để tiết kiệm thời gian gán nhãn; đổi lại, phần kiểm tra dữ liệu và chia tập phải làm chặt hơn. Các phần còn lại của đề xuất (YOLO26n làm baseline, so sánh YOLO26s, ByteTrack, FastAPI, nguyên tắc không chọn mô hình hay ngưỡng trên tập Test) được giữ nguyên.

| Hạng mục | v1.0 | v1.1 |
|---|---|---|
| Nguồn dữ liệu | Tự thu video, tự gán nhãn bounding box | Hai bộ ảnh công khai đã có nhãn (CC BY 4.0); tự quay một vài video riêng chỉ để kiểm thử tracking và đếm |
| Gán nhãn | CVAT/Roboflow/LabelImg, review kép | Không gán mới; thay bằng kiểm tra chất lượng nhãn có sẵn (mẫu khoảng 50 ảnh mỗi nguồn) và chuẩn hóa tên lớp |
| Chia tập | Theo video hoặc lô sản xuất | Theo nhóm (group): video ID với bộ A, cụm ảnh gần trùng với bộ B; phân tầng theo nguồn và theo lớp |
| Quy mô | Chưa xác định | 1.000 ảnh gốc (780 + 220), dự kiến khoảng 700 / 150 / 150 |
| Tiến độ | Tuần 2–3 dành cho thu thập và gán nhãn | Tuần 2 dành cho tải, kiểm tra, chia tập; thời gian tiết kiệm được chuyển sang tracking, đếm và giao diện |
| Giao diện | Dashboard cơ bản | Thêm đường đếm ảo, bộ đếm đồng bộ với video, biểu đồ pass/defect theo thời gian, gallery chai lỗi |

# 2. Nguồn dữ liệu

| Thuộc tính | Bộ A | Bộ B |
|---|---|---|
| Tên trên Roboflow Universe | Bottle defect detection | bottle_defects |
| Tác giả | Defect detection | robo |
| Đường dẫn | https://universe.roboflow.com/defect-detection-u5n4a/bottle-defect-detection-nmecq | https://universe.roboflow.com/robo-lupdk/bottle_defects-tpmcq |
| Số ảnh | 780 | 220 |
| Lớp gốc | good, defect | Good, defective |
| License | CC BY 4.0 | CC BY 4.0 |
| Dạng tên file | IMG_####_MOV-00## (gợi ý frame cắt từ video quay bằng điện thoại) | good_bottle (n), Defective (n) (ảnh đánh số theo lớp, không có dấu hiệu là frame video) |
| Ghi chú | Có ảnh chứa cả hai lớp (nhiều chai trong một ảnh) | Trang dự án chưa có phiên bản dataset nào được tạo |

License CC BY 4.0 yêu cầu ghi công; thông tin trích dẫn năm ở Phụ lục A.

## 2.1. Giới hạn cần nói rõ

- Thông tin ở bảng trên lấy từ trang mô tả và tên file; nội dung ảnh chưa được xem toàn bộ. Cần xác nhận sau khi tải (mục 2.2).
- Hai bộ do hai nhóm khác nhau thực hiện, nên định nghĩa lỗi và điều kiện chụp có thể khác nhau.
- Chưa biết số bounding box mỗi lớp. Nếu lớp defect ít, Recall của lớp này sẽ dao động mạnh.
- Tổng 1.000 ảnh là quy mô nhỏ; kết quả chỉ đại diện cho điều kiện của hai bộ này, không tuyên bố tổng quát cho mọi dây chuyền.

## 2.2. Việc xác nhận ngay sau khi tải

1. Tổng số ảnh bằng 780 + 220 = 1.000 (nếu nhiều hơn, bản tải về đang chứa ảnh augmentation).

2. Danh sách lớp thực tế trong data.yaml của từng bộ.
3. Số box mỗi lớp theo từng nguồn.
4. Số video ID khác nhau ở bộ A (quyết định việc chia theo nhóm có đủ nhóm hay không).
5. Xem khoảng 50 ảnh mỗi nguồn: ảnh có phải sản phẩm trên nền băng chuyền hay không, box có đúng hay không, ảnh có nhiễu hoặc trùng nhiều hay không.

# 3. Chuẩn hóa nhãn

| Lớp gốc | Nguồn | Lớp chuẩn | Class ID |
|---|---|---|---|
| good | A | pass | 0 |
| Good | B | pass | 0 |
| defect | A | defect | 1 |
| defective | B | defect | 1 |

Giữ nguyên bounding box, chỉ đổi class ID. data.yaml chuẩn có names: 0 pass, 1 defect. Nếu phát hiện lớp lạ hoặc ảnh không có nhãn, loại khỏi kho và ghi lại số lượng trong data card.

# 4. Quy trình chia tập hợp lệ

## 4.1. Các nguồn rò rỉ dữ liệu cần tránh

- Frame liền kề của cùng một video nằm ở cả Train và Test (bộ A).
- Ảnh chụp lặp lại cùng một chai nằm ở nhiều tập (bộ B).
- Bản augmentation của cùng một ảnh nằm ở các tập khác nhau.
- Cách chia gốc của Roboflow là chia theo từng ảnh, nên không dùng lại.

## 4.2. Nguyên tắc

1. Đơn vị chia là nhóm (group), không phải ảnh: mọi ảnh của một nhóm vào cùng một tập.
2. Phân tầng theo nguồn và theo việc ảnh có chứa defect hay không, để mỗi tập đều có cả hai nguồn và cả hai lớp.
3. Hạt giống cố định, kết quả chia được lưu thành manifest và đưa vào git.
4. Tập Test không được dùng để chọn mô hình, ngưỡng hay cấu hình tracker.
5. Khi kiểm tra thất bại, sửa quy tắc nhóm hoặc hạt giống; không chuyển ảnh bằng tay.

## 4.3. Các bước

1. Tải cả hai bộ ở định dạng YOLO, chọn phiên bản không augmentation và không resize. Gộp các thư mục train/valid/test của Roboflow thành một kho chung theo từng nguồn (data/raw/A và data/raw/B).
2. Kiểm tra tổng số ảnh bằng 1.000 (mục 2.2).
3. Chuẩn hóa nhãn theo mục 3.
4. Lập manifest, mỗi ảnh một dòng (cột ở Phụ lục B).
5. Gán nhóm.
   - Bộ A: nhóm là video ID lấy từ tên file (phần IMG_#### trước _MOV). Ảnh nào không tách được ID thì rơi về bước gom ảnh gần trùng như bộ B.
   - Bộ B: gom ảnh gần trùng bằng perceptual hash, ngưỡng khoảng cách ban đầu 6 và điều chỉnh sau khi xem mẫu. Nếu quan sát cho thấy ảnh có số thứ tự liền kề thường là cùng một chai, gộp theo khối ảnh liên tiếp (ví dụ 10 ảnh một khối).
6. Chia theo nhóm có phân tầng, hai lượt: lượt đầu tách tập Test, lượt sau tách tập Validation từ phần còn lại. Nhãn phân tầng là tổ hợp nguồn và có-defect-hay-không.
7. Chạy bộ kiểm tra hợp lệ (mục 4.5).
8. Khóa kết quả: lưu manifest.csv, hạt giống, phiên bản thư viện; commit vào repo. Tập Test chỉ được mở khi đã khóa mô hình, ngưỡng và cấu hình tracker.
9. Tạo data/data.yaml trỏ tới ba thư mục images/labels của Train, Val, Test.

## 4.4. Tỷ lệ và kích thước dự kiến

| Tập | Tỷ lệ tham khảo | Số ảnh ước tính | Công dụng |
|---|---|---|---|
| Train | khoảng 71% | khoảng 710 | Học tham số mô hình; augmentation do Ultralytics áp dụng khi huấn luyện |
| Validation | khoảng 14% | khoảng 145 | So sánh YOLO26n với YOLO26s, chọn checkpoint và confidence threshold |
| Test | khoảng 14% | khoảng 145 | Đánh giá cuối cùng một lần sau khi khóa cấu hình |

Vì chia theo nhóm, tỷ lệ thực tế sẽ lệch nhẹ so với bảng. Nếu bộ A chỉ có rất ít video (ví dụ dưới 10 nhóm), độ lệch có thể lớn hơn: chấp nhận độ lệch và ghi lại, không chuyển sang chia theo từng ảnh.

## 4.5. Điều kiện kiểm tra hợp lệ

- Không có nhóm nào xuất hiện ở hơn một tập (kiểm tra bằng test tự động trong tests/test_split.py).
- Không có cặp ảnh gần trùng (khoảng cách hash nhỏ hơn ngưỡng) nằm ở hai tập khác nhau.
- Tổng ảnh là 1.000 sau khi loại ảnh lỗi; số ảnh bị loại được ghi lại.
- Mỗi tập có đủ cả hai nguồn và cả hai lớp.
- Tỷ lệ box defect của từng tập được lập bảng và so với toàn bộ.
- Số box defect trong Test đủ để báo cáo Recall: quy ước thực hành là khoảng 30 box trở lên; ít hơn thì kết luận về Recall chỉ mang tính tham khảo và phải nêu rõ.

## 4.6. Cách báo cáo để không nói quá

- Lập bảng thống kê: số ảnh, số nhóm, số box pass và defect cho từng tập và từng nguồn.
- Với Precision và Recall của lớp defect, báo cáo kèm khoảng tin cậy 95% (ví dụ Wilson) hoặc bootstrap theo nhóm, vì số mẫu defect có hạn.
- Không kết luận một mô hình tốt hơn mô hình kia nếu chênh lệch nằm trong khoảng tin cậy.
- Báo cáo kết quả riêng cho từng nguồn trong tập Test, ngoài kết quả gộp.

## 4.7. Cách dùng hai nguồn cho thí nghiệm độ bền

Ngoài lần chia chính (hai nguồn trộn trong cả ba tập), có một thí nghiệm bổ sung: huấn luyện chỉ trên phần Train của bộ A, rồi đánh giá trên toàn bộ bộ B, vốn chưa từng được mô hình thấy. Thí nghiệm này đo khả năng chuyển sang nguồn ảnh khác, và không được dùng để chọn ngưỡng hay mô hình.

## 4.8. Khung mã tham khảo

Đây chỉ là khung bắt đầu, cần chạy thử và chỉnh theo dữ liệu thật.

```
import re
import pandas as pd
import imagehash
from PIL import Image
from sklearn.model_selection import StratifiedGroupKFold

def video_id(name):  # bộ A: IMG_9974_MOV-0011... -> IMG_9974
    m = re.match(r'(IMG_[0-9]+)_MOV', name)
    return m.group(1) if m else None

def hash_clusters(paths, max_dist=6):  # gom ảnh gần trùng (cho bộ B)
```

```
    hs = [imagehash.dhash(Image.open(p)) for p in paths]
    parent = list(range(len(paths)))
    def find(i):
        while parent[i] != i:
            parent[i] = parent[parent[i]]
            i = parent[i]
        return i
    for i in range(len(hs)):
        for j in range(i + 1, len(hs)):
            if hs[i] - hs[j] <= max_dist:
                parent[find(i)] = find(j)
    return ['B:' + str(find(i)) for i in range(len(paths))]

# df: mỗi dòng một ảnh, cột: path, source, group, has_defect (0/1)
df['y'] = df['source'] + '_' + df['has_defect'].astype(str)

sgkf = StratifiedGroupKFold(n_splits=7, shuffle=True, random_state=42)
tv_idx, te_idx = next(sgkf.split(df, df['y'], df['group']))
test = df.iloc[te_idx]
trainval = df.iloc[tv_idx].reset_index(drop=True)

sgkf2 = StratifiedGroupKFold(n_splits=6, shuffle=True, random_state=42)
tr_idx, va_idx = next(sgkf2.split(trainval, trainval['y'],
trainval['group']))
train, val = trainval.iloc[tr_idx], trainval.iloc[va_idx]

# kiểm tra: không nhóm nào ở hai tập
assert not set(train['group']) & set(val['group'])
assert not set(train['group']) & set(test['group'])
assert not set(val['group']) & set(test['group'])
```

## 5. Ma trận thí nghiệm điều chỉnh

| Mã | Cấu hình | Câu hỏi / đầu ra |
|---|---|---|
| E0 | Baseline không dự đoán đối tượng | Kiểm tra metric và pipeline; Recall defect bằng 0 theo định nghĩa |
| E1 | YOLO26n, checkpoint pretrained, split cố định ở mục 4 | Baseline hiệu năng và tốc độ |
| E2 | YOLO26s, cùng split, cùng imgsz và quy tắc chọn ngưỡng | Lợi ích bổ sung so với YOLO26n, kèm khoảng tin cậy |

| Mã | Cấu hình | Câu hỏi / đầu ra |
|---|---|---|
| E3 | Cố định mô hình tốt nhất, quét confidence trên Validation | Bảng Precision, Recall, FN, FP và ngưỡng đề xuất |
| E4 | Cố định detector và ByteTrack; chạy trên video tự quay | Sai số số đếm so với đếm tay; mất ID, ID bị tách |
| E5a | Kết quả theo từng nguồn trong tập Test | Mô hình có lệch về một nguồn không |
| E5b | Huấn luyện chỉ trên Train của bộ A, đánh giá trên toàn bộ bộ B | Khả năng chuyển nguồn ảnh (mục 4.7) |

Tập video kiểm thử cho E4 gồm 3–5 video, mỗi video 30–60 giây, tự quay và không thuộc 1.000 ảnh. Ít nhất một video không được dùng để chỉnh bất kỳ tham số nào (ngưỡng, vị trí đường đếm). Nếu không có chai lỗi thực tế để quay, E4 chỉ đánh giá số đếm chai tổng, còn phần pass/defect trên video chỉ minh họa định tính và phải nêu rõ giới hạn này.

# 6. Hướng phát triển và cách phát triển

## 6.1. Ba lớp mục tiêu

1. **Lõi (bắt buộc):** pipeline dữ liệu hợp lệ, YOLO26n và YOLO26s, chọn ngưỡng trên Validation, tracking và đếm, Web API (upload, job, kết quả, CSV/JSON).
2. **Trình diễn (đã chọn):** đường đếm ảo vẽ trên video; bộ đếm đồng bộ với thời điểm phát video; biểu đồ pass/defect theo thời gian; gallery các chai lỗi kèm confidence, bấm vào ảnh thì video nhảy tới đúng thời điểm.
3. **Mở rộng sau MVP:** nhận luồng camera trực tiếp (RTSP), thêm loại sản phẩm khác (mỗi loại có lớp pass và defect riêng), phân loại loại lỗi, segmentation khi cần khoanh vùng lỗi, xuất ONNX hoặc TensorRT, cảnh báo.

Về từ luồng video trong tên đề tài: MVP nhận file video và đọc tuần tự từng frame; xử lý luồng trực tiếp thuộc lớp 3. Cần xác nhận với giảng viên xem cách hiểu này có đủ không.

## 6.2. Cách phát triển

- **Làm từng lát cắt dọc:** có bản chạy được từ upload đến kết quả với mô hình pretrained trước, sau đó thay bằng trọng số đã fine-tune.
- **Viết test cho phần dễ sai:** logic đếm băng đường ảo (track đi qua, đứng yên, đi qua rồi quay lại) và kiểm tra rò rỉ khi chia tập.
- **Cấu hình nằm ngoài code:** vị trí đường đếm, ngưỡng, cấu hình tracker lưu trong file YAML và được ghi lại cho mỗi job.

- **Mỗi thí nghiệm có ID riêng:** lưu config, hạt giống, phiên bản thư viện, phần cứng; không ghi đè kết quả cũ.
- **Git:** commit manifest, config và mã; không commit dữ liệu lớn hay file nhạy cảm.
- **Quy tắc đóng băng:** sau khi khóa mô hình, ngưỡng và tracker, không chỉnh lại sau khi đã nhìn kết quả Test.

### 6.3. Đầu ra của worker

Với mỗi job, worker ghi: video có khung và đường đếm (mã hóa H.264 để trình duyệt phát được), events.json (track ID, lớp, confidence, thời điểm, frame), summary.json, và thư mục ảnh cắt các chai lỗi. Lớp cuối cùng của một track được quyết định theo quy tắc gộp trên toàn track (ví dụ defect nếu có ít nhất N frame defect vượt ngưỡng), vì một chai có thể bị gán lớp khác nhau ở các frame khác nhau; N được chọn trên Validation hoặc video chỉnh tham số, không chọn trên Test.

Endpoint bổ sung so với v1.0: video kết quả, events, và ảnh cắt theo track ID của từng job.

## 7. Các bước thực thi

| Tuần | Công việc chính | Sản phẩm bàn giao |
|---|---|---|
| 2 (hiện tại) | Tải hai bộ và kiểm tra (mục 2.2); chuẩn hóa nhãn; lập manifest, gán nhóm, chia tập, chạy kiểm tra hợp lệ; song song viết module đếm băng đường ảo kèm test và quay 3–5 video kiểm thử | manifest.csv, data.yaml, data card, báo cáo thống kê lớp/box, bộ kiểm tra chia tập, module đếm có test |
| 3 | Huấn luyện thử YOLO26n vài chục epoch rồi chạy đầy đủ; phân tích lỗi trên Validation; huấn luyện YOLO26s cùng split; dựng khung FastAPI (health, upload, trạng thái job) chạy với mô hình pretrained | best.pt của E1, bảng kết quả Validation, API khung |
| 4 | So sánh YOLO26n và YOLO26s; quét ngưỡng trên Validation; khóa mô hình, ngưỡng, tracker; nối tracking và đếm vào worker; xuất video overlay, events, ảnh cắt | Mô hình và cấu hình đã khóa, bảng ngưỡng, worker chạy end-to-end |
| 5 | Giao diện (video, bộ đếm đồng bộ, biểu đồ, gallery), xuất CSV/JSON; kiểm tra file upload, timeout, xóa file tạm | Web demo end-to-end, tài liệu API |

| Tuần | Công việc chính | Sản phẩm bàn giao |
|---|---|---|
| 6 | Chạy Test một lần; E4 trên video; E5a, E5b; đo latency; hoàn thiện báo cáo, model card, README, slide | Báo cáo cuối, model card, bộ demo |

### Điểm kiểm tra

- Cuối tuần 2: giảng viên hoặc người hướng dẫn xem data card, quy tắc chia tập và định nghĩa lớp đã chuẩn hóa.
- Cuối tuần 3: kiểm tra lại rò rỉ dữ liệu và chất lượng nhãn trước khi chốt các thí nghiệm.
- Cuối tuần 4: review metric cùng các ví dụ false negative trước khi khóa cấu hình và tích hợp web.
- Cuối tuần 6: demo trên video chưa dùng để chỉnh tham số; trình bày giới hạn rõ ràng.

## 8. Rủi ro cập nhật

| Rủi ro | Tác động | Giảm thiểu |
|---|---|---|
| Lớp defect ít, tập Test nhỏ | Recall dao động mạnh, khó so sánh hai mô hình | Báo cáo khoảng tin cậy; nêu số box defect mỗi tập; không kết luận khi chênh lệch nằm trong khoảng tin cậy |
| Hai nguồn định nghĩa defect khác nhau | Mô hình học nhãn không nhất quán | Kiểm tra mẫu nhãn mỗi nguồn; báo cáo riêng từng nguồn; nếu lệch nhiều thì dùng bộ B chỉ làm tập kiểm tra độ bền |
| Bộ A có ít video | Chia theo nhóm cho tỷ lệ lệch | Chấp nhận độ lệch và ghi lại; không chia theo ảnh |
| Ảnh gần trùng ở bộ B không phát hiện hết | Kết quả Test lạc quan | Kiểm tra băng hash và xem mẫu bằng mắt; điều chỉnh ngưỡng hash |
| Ảnh dataset là ảnh tĩnh, không phải video | Không đánh giá được tracking từ dữ liệu này | Dùng video tự quay riêng cho E4 và nêu rõ phạm vi |
| Thiếu chai lõi thật để quay video | E4 không kiểm được pass/defect trên video | Chỉ đánh giá số đếm tổng; pass/defect trên video chỉ để minh họa định tính |

| Rủi ro | Tác động | Giảm thiểu |
|---|---|---|
| License và ghi công | Vi phạm điều kiện CC BY 4.0 | Ghi nguồn và trích dẫn hai bộ trong báo cáo và README |

# Phụ lục A. Trích dẫn dữ liệu

```
@misc{ bottle-defect-detection-nmecq_dataset,
  title = { Bottle defect detection Dataset },
  type = { Open Source Dataset },
  author = { Defect detection },
  url = { https://universe.roboflow.com/defect-detection-u5n4a/bottle-defect-
detection-nmecq },
  journal = { Roboflow Universe },
  publisher = { Roboflow },
  year = { 2025 },
  month = { aug }
}

@misc{ bottle_defects-tpmcq_dataset,
  title = { bottle_defects Dataset },
  type = { Open Source Dataset },
  author = { robo },
  url = { https://universe.roboflow.com/robo-lupdk/bottle_defects-tpmcq },
  journal = { Roboflow Universe },
  publisher = { Roboflow },
  year = { 2025 },
  month = { nov }
}
```

# Phụ lục B. Các cột của manifest

sample_id, source (A hoặc B), original_class_names, image_path, label_path, group_id, n_pass_boxes, n_defect_boxes, has_defect, image_hash, split (train, val hoặc test), ghi chú review nhãn.